From-Scratch Decision Trees and Pegasos SVM for Bina.az Real-Estate Modelling

Team Member 1 Team Member 2 Team Member 3

MLE-AI-201 Machine Learning, AI Academy

***Abstract*— We study 100,775 scraped Bina.az sale records as two supervised learning problems: sale-price regression and a derived premium-versus-standard classification task. After keeping the latest snapshot of repeated listings and applying leakage-free cleaning, 64,128 unique listings and 29 numeric features remained. We implemented a CART-style decision tree entirely in NumPy with Gini, entropy, and MSE criteria, as well as a soft-margin SVM trained from scratch with Pegasos sub-gradient updates. A random Fourier feature (RFF) mapping was added as the required non-linear SVM extension. Hyperparameters were selected on validation data only, while the premium threshold (210,000 AZN) and standardization statistics were learned from training data only. On the held-out test set, our tree classifier reached F1=0.9099 and ROC-AUC=0.9630, almost identical to scikit-learn's tree (F1=0.9103). Our linear Pegasos model reached F1=0.8228 and ROC-AUC=0.9083. The from-scratch regression tree obtained RMSE=310,716 AZN and $R^2 = 0.592$. Results show that the tree captures the non-linear structure of tabular real-estate data substantially better than the linear margin model, while rare property types and luxury outliers remain difficult. Repository: `https://github.com/Hasanov57/Bina.az_ML_Final_Project`.**

***Index Terms*—**decision tree, SVM, Pegasos, random Fourier features, regression, classification, real estate.

I. Introduction

Real-estate price modelling is a useful test of machine-learning fundamentals because prices depend on interacting structural and geographic variables and real listing data is noisy. This project uses a Bina.az sale dump to study two tasks: predicting sale price and predicting whether a listing belongs to a premium price tier. Our main contributions are: (1) a reproducible cleaning and feature pipeline, (2) a from-scratch NumPy decision tree supporting regression and classification, (3) a from-scratch soft-margin SVM trained with Pegasos plus a random Fourier feature extension, and (4) matched comparisons against scikit-learn with validation curves, uncertainty estimates, feature analysis, and error analysis.

II. Background

A decision tree recursively chooses a feature and threshold that maximally decreases node impurity. For classification we implement Gini impurity and entropy; for regression we use variance/MSE reduction. Leaves predict the majority class or the mean target value. Tree complexity is controlled by maximum depth, minimum split size, minimum leaf size, and minimum impurity decrease.

For binary classification, the soft-margin SVM minimises regularised hinge loss. We use the Pegasos stochastic sub-gradient method [1] rather than a QP/SMO solver. At update step t, the learning rate is $\eta_t = 1/(\lambda t)$. If a sampled point violates the margin, its labelled feature vector contributes to the update; otherwise only regularisation shrinkage is applied. Features are standardised from training statistics. For non-linearity, we approximate an RBF kernel with random Fourier features (RFF) and apply the same linear Pegasos solver in the mapped space.

III. Data and Preprocessing

The raw CSV contains 100,775 rows and 51 columns. Many rows are repeated snapshots of the same listing. We sort snapshots by scrape time and retain the latest record per listing URL, then apply fixed plausibility filters for obvious entry/scrape errors (for example 11 AZN listings, 600M AZN listings, and 200,000 m^2 area values). This produces 64,128 unique cleaned listings.

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows	100,775
Clean unique listings	64,128
Features	29
Train / Val / Test	44,890 / 9,619 / 9,619
Premium threshold	210,000 AZN

We parse floor area, room count, current/total floor, land area, latitude and longitude into numeric values. Missing room/floor/land values are encoded as zero with explicit missing indicators, avoiding medians learned from validation/test data. Low-cardinality category and city columns are one-hot encoded. Repair, deed and mortgage availability are encoded as binary variables. High-cardinality identifiers, raw URLs, names, addresses and free text are not used in the core model. Critically, `unit_price` and `total_price` are removed because they are arithmetic functions of the target and would cause target leakage.

We first select the 70% training split. The price-tier threshold is then the median of training prices only: 210,000 AZN. The remaining observations are divided into 15% validation and 15% test sets with tier stratification. The final split sizes are 44,890 / 9,619 / 9,619, with premium proportions 0.496 / 0.494 / 0.494. Scaling for the SVM uses training means and standard deviations only.

IV. Method

A. Decision Tree

For each numeric feature we sort values and evaluate candidate thresholds between adjacent distinct values. The best split maximises

$$\Delta I = I(P) - \frac{N_L}{N} I(L) - \frac{N_R}{N} I(R). \quad (1)$$

Classification impurity is either $1 - \sum_k p_k^2$ (Gini) or $-\sum_k p_k \log_2 p_k$ (entropy). Regression impurity is node variance. The implementation recursively creates child nodes until a stopping condition is met. Leaf class probabilities are empirical class frequencies, and feature importance is accumulated weighted impurity decrease. A toy deterministic dataset is checked against scikit-learn; the root feature, threshold and predictions match.

B. Pegasos SVM and RFF

The linear SVM minimises

$$J(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)). \quad (2)$$

Labels are mapped to $\{-1, +1\}$. A constant feature folds the bias into the update, followed by the standard Pegasos L2-ball projection. We track the regularised hinge objective during training. Support vectors are training observations satisfying $y_i f(x_i) \leq 1$. The RFF extension samples random frequencies corresponding to an RBF kernel and maps each input to cosine features before applying Pegasos.

V. Experimental Setup

Regression trees are trained on $\log(1 + price)$ because the price distribution is strongly right-skewed; predictions are exponentiated for price-scale RMSE, MAE and R^2. Classification uses precision, recall, F1 and ROC-AUC. All random seeds are fixed to 42.

Validation data only is used to select hyperparameters. Regression depth is swept over 4, 6, 8, 10 and 12, followed by minimum-leaf sizes 5, 20, 50 and 100. Classification compares Gini against entropy, then sweeps depth and minimum-leaf size. Pegasos sweeps $\lambda \in \{10^{-2}, 3 \times 10^{-3}, 10^{-3}, 3 \times 10^{-4}\}$, epochs $\in \{1, 2, 3, 5\}$, and three learning-rate schedules (Pegasos $1/(\lambda t)$, inverse-square-root, and constant). RFF sweeps $\gamma \in \{0.03, 0.1, 0.3\}$. The selected configurations are regression tree depth 12/leaf 5, classification tree Gini/depth 11/leaf 5, linear SVM $\lambda = 10^{-3}$ with 3 epochs and the Pegasos schedule, and RFF $\gamma = 0.1$ with 120 components. Baselines are scikit-learn decision trees and a `StandardScaler + SGDClassifier(loss='hinge')` pipeline.

VI. Results

Our regression tree has MAE=87,181 AZN and $R^2 = 0.5921$, compared with 86,475 AZN and $R^2 = 0.6215$ for the scikit-learn tree. For classification, the from-scratch tree is nearly indistinguishable from the library implementation on F1/AUC, indicating that split search, stopping rules and leaf predictions are behaving correctly. The linear SVM is weaker, consistent with a problem where location, area and property type interact non-linearly. The RFF extension did not improve F1 under the tested dimensionality/bandwidth settings.

Table 2. Held-out test results.

Model	RMSE	F1	AUC
Our Tree	310,716	0.9099	0.9630
Sklearn Tree	299,329	0.9103	0.9629
Our Linear Pegasos	–	0.8228	0.9083
Our RFF Pegasos	–	0.8101	0.8936
Sklearn SGD Hinge	–	0.8259	0.9077

Table 3. Fit and prediction timing on the final split.

Model	Fit (s)	Predict (ms/1000)
Our Tree Classifier	0.96	3.51
Sklearn Tree Classifier	0.18	0.19
Our Linear Pegasos	0.65	0.03
Our RFF Pegasos	0.82	2.35
Sklearn SGD pipeline	0.18	0.24

Three-fold cross-validation on a fixed-seed 18,000-row subset of the development data provides a practical uncertainty estimate without using the final test set. Mean $\pm$ standard deviation F1 is 0.8949 ± 0.0054 for our tree, 0.8970 ± 0.0062 for the sklearn tree, 0.8216 ± 0.0096 for linear Pegasos, 0.8099 ± 0.0031 for RFF Pegasos, and 0.8324 ± 0.0013 for sklearn's hinge-loss pipeline. Regression RMSE is $364,775 \pm 21,116$ AZN for our tree and $364,667 \pm 21,064$ AZN for the sklearn tree. The subset is used only to keep repeated from-scratch tree fitting computationally practical.

The SVM controls have visible effects. With $\lambda = 10^{-3}$ and the Pegasos schedule, validation F1 changes from 0.8224 after one epoch to 0.8330 after three epochs, while the margin width changes from 0.241 to 0.335. At three epochs, Pegasos, inverse-square-root, and constant schedules obtain F1 values 0.8330, 0.8000, and 0.8204 with margin widths 0.335, 0.789, and 0.437 respectively. Thus the epoch count and learning-rate schedule change both the learned separator and its margin.

A separate fixed-seed scaling experiment with 2k, 5k and 10k training rows shows fit time increasing with sample size for both models; the from-scratch tree grows from about 0.10 s to 0.29 s, while linear Pegasos grows from about 0.03 s to 0.14 s.

VII. Discussion and Limitations

The tree relies most heavily on area (58.8% of impurity-decrease importance), longitude (19.6%) and latitude (9.8%), confirming that both property size and location dominate the tier decision. Room count, new-build status, land area and floor information contribute smaller gains. Tree accuracy is high for new and old apartments (about 92%) but lower for land (81.8%) and commercial objects (84.6%), which are more heterogeneous and have fewer observations.

Regression error is strongly concentrated among expensive properties. For the top 5% of test prices, MAE is about 773,000 AZN, versus about 49,500 AZN for the remaining

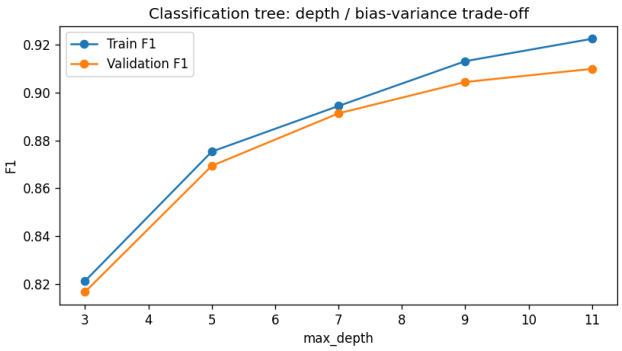


Figure 1. Increasing depth improves training F1 continuously while validation improvement becomes smaller, showing the beginning of the bias-variance trade-off.

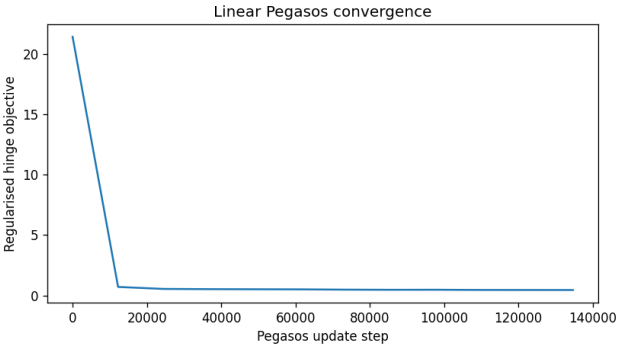


Figure 2. Pegasos regularised hinge objective decreases strongly overall, with expected stochastic fluctuations.

95%. This explains why the heavy-tailed price distribution produces a large RMSE even after log-target modelling. The fixed cleaning rules remove obvious errors but cannot identify every unrealistic listing. We intentionally exclude description text and raw address to keep the core implementation manageable; NLP/address features could improve future models. Rare city one-hot variables also produce unstable-looking SVM weights after standardisation, so individual rare-category coefficients should not be over-interpreted.

The RFF result shows that adding a non-linear mapping does not automatically improve generalisation: with 120 random features, it approximately matches but does not beat the linear Pegasos model. A larger feature map and a broader joint search over γ, dimensionality and λ could improve it, at increased computational cost.

VIII. Conclusion

We implemented and evaluated decision-tree and SVM algorithms from first principles on a messy Azerbaijani real-estate dataset. The from-scratch tree closely matches scikit-learn for price-tier classification and clearly outperforms the linear margin model, while price regression remains sensitive to luxury outliers. The experiments demonstrate the practical importance of leakage control, train-only preprocessing statistics, validation-based tuning, feature scaling and error analysis in addition to algorithm implementation.

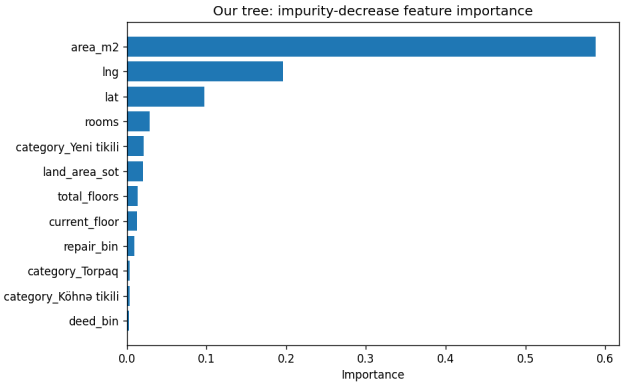


Figure 3. Impurity-decrease importance of the final from-scratch classification tree.

Tools and Acknowledgements

The dataset is the Bina.az Sale Project published on Kaggle by Sehriyar Memmedli. NumPy and pandas were used for data/array operations; matplotlib for figures; scikit-learn only for comparison baselines. AI assistance was used during development to propose implementation approaches, draft portions of code, explain algorithms, help debug, and refine report wording. The team is responsible for reviewing, testing, understanding and defending the submitted implementation and results.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.